

X Algorithm: A Comprehensive Research Report

Architecture, Signal Mechanics, Engagement Modeling, and Algorithmic Optimization
Framework

June 2026

X Algorithm

A Comprehensive Research Report

Architecture, Signal Mechanics, Engagement Modeling,
and Algorithmic Optimization Framework

*Based on the `xai-org/x-algorithm` Repository:
January & May 2026 Releases*

Emil Veliyev

`github.com/emillvl`

Source: `github.com/xai-org/x-algorithm`

License: Apache 2.0

Release: May 15, 2026 (commit `e414c17` , 187 files, ~18,000 lines)

Author

Emil Veliyev

Report Date

June 2026



Version Note: This report is grounded in two canonical source releases: the **January 20, 2026** initial open-source commit of `xai-org/x-algorithm` and the **May 15, 2026** major update (commit `e414c17`, 187 files changed, ~18,000 lines of new code). Where specific implementation details are taken directly from repository source files, they are cited as such. Where analysis is derived from field observations or from the legacy `twitter/the-algorithm` (2023) release, this distinction is made explicit.

Table of Contents

1. Introduction: Purpose and Scope
2. Repository Structure and Architectural Components
 - 2.1 Module Overview and Language Composition
 - 2.2 Thunder: Real-Time In-Network Post Store
 - 2.3 Phoenix Retrieval: Two-Tower Architecture and ANN Search
 - 2.4 The GraphJet Legacy and Its Transformer Successor
 - 2.5 Candidate Isolation: `make_recsys_attn_mask` and Complexity Reduction
 - 2.6 Grox: The Content-Understanding Pipeline
 - 2.7 Home Mixer: The Orchestration Layer
3. Scoring Formula and Engagement Modeling
 - 3.1 The Linear-Weighting Fallacy
 - 3.2 Phoenix Ranker: Multi-Head Non-Linear Scoring
 - 3.3 The Fifteen-to-Nineteen Engagement Probability Heads
 - 3.4 Weighted Scorer and Final Score Computation
4. Positive Signals: Actions That Drive Reach and Discovery
 - 4.1 Dwell Time Maximization
 - 4.2 Media Thumb-Stop, Hold Rate, and the Banger Screen
 - 4.3 Direct Message Shares
 - 4.4 Bookmarks and Quote Tweets
 - 4.5 Profile Visit-to-Follow Conversion
 - 4.6 Self-Reply Bumping and Early Reply Velocity
 - 4.7 Topical Authority and Embedding Stability
 - 4.8 X Premium Effect on In-Network and Out-of-Network Visibility
5. Negative Signals: Actions That Suppress Reach and Trigger Penalties
 - 5.1 External Links
 - 5.2 Hashtag Inflation
 - 5.3 Burst Posting and Author Diversity Decay
 - 5.4 Grox Sentiment Loop Penalty (Conversational Gaming)
 - 5.5 Grox Toxicity and Brand Safety Filter
 - 5.6 Sub-1,000 Follower Account Filter: The `lte_1000` Bucket
 - 5.7 Unfollow Spikes and Follow/Follower Ratio Imbalance
 - 5.8 Visual Recycling and the Banger `slop_score` Penalty
 - 5.9 YouTube and External Video Player Links
6. Temporal and Complexity Analysis

7. Security, Adversarial Risk, and the Open-Source Paradox
 8. Reproducibility Framework
 9. Conclusion
-

1. Introduction: Purpose and Scope

X (formerly Twitter) operates one of the highest-throughput real-time information-ranking systems in the history of software engineering. By 2026, the platform processes approximately **500 million posts per day** and executes roughly **5 billion individual ranking decisions** across its user base within the same 24-hour window. Every time a user opens the “For You” feed, a gesture that lasts fractions of a second, the system must answer an existential computational question: out of approximately one billion posts written in the past several days, which fifty should this person see, delivered in under 200 milliseconds?

The engineering philosophy that animates the answer to this question underwent a seismic shift between 2023 and 2026. The original 2023 system, released under `twitter/the-algorithm`, was a **Scala-based architecture** assembled from hundreds of hand-engineered features, discrete numerical signals, manually tuned weighting tables, topic-affinity clusters, and PageRank-derived reputation scores. Engineers defined every feature and adjusted every weight by hand; what the system “knew” about relevance was constrained entirely by what humans could formalize and express in code.

That philosophical approach was abandoned wholesale in the transition that began in late 2025 and culminated in the January 2026 open-source release at `github.com/xai-org/x-algorithm`. In October 2025, Elon Musk announced the full integration of Grok into the recommendation pipeline: “*Grok will literally read every post and watch every video, 100 million-plus per day, to match users with content they’re most likely to find interesting.*” The January 2026 commit confirmed this transition. The repository README states it plainly: “**We have eliminated every single hand-engineered feature and most heuristics from the system.**”

The new architecture is written **62.9% in Rust** and **37.1% in Python**, representing a complete rewrite from Scala. Rust handles the latency-critical real-time services (in-memory post storage, orchestration, pipeline framework), while Python (with JAX) handles the machine-learning inference components. The resulting system is not an incremental improvement over what came before, it is a categorical architectural replacement with fundamentally different epistemological premises about how relevance is discovered.

This report constitutes a comprehensive technical and practical analysis of that system as of the **May 15, 2026 release** (the largest single update to the repository, shipping a unified end-to-end runnable inference pipeline for the first time). Its purpose is to document every mechanistic layer, from the Kafka event stream that ingests posts at millisecond granularity, through the two-tower vector retrieval that discovers out-of-network content, through the Grok-based transformer ranker that predicts nineteen engagement probabilities, through the Grox content-safety filter, to the final blending step that weaves organic content with advertisements, and to connect each layer to its concrete consequences for creators, marketers, and platform researchers.

2. Repository Structure and Architectural Components

2.1 Module Overview and Language Composition

The `xai-org/x-algorithm` repository is organized into five primary service modules and one shared-framework crate:

```
x-algorithm/
├─ home-mixer/          # Rust - Orchestration, gRPC server, ad blending
├─ thunder/             # Rust - Real-time in-memory in-network post store
├─ candidate-pipeline/  # Rust - Generic pipeline trait framework
├─ phoenix/             # Python/JAX - Two-tower retrieval + transformer ranker
└─ grox/                # Python - Vision-Language Model content understanding
```

Each module has a clearly defined responsibility within the overall pipeline, and no module encroaches on another’s domain. This separation is not merely organizational; it reflects distinct latency and computational requirements. Rust services are expected to respond within single-digit milliseconds; Python/-JAX inference operates in the tens-to-hundreds of milliseconds range and is isolated to pre-computed embeddings and ranked candidates.

The `home-mixer/` module excludes three sub-directories from the open-source release: `clients/`, `params/`, and `util/`, marked in the source with `pub mod clients; // Excluded` comments. Notably, `params/` contains the exact numerical weights used in production, these were deliberately withheld from the public release. The implication is significant: the architecture and all signal definitions are public, but the precise production weight values are not. Any specific numerical engagement weights cited by analysts (such as “a reply is worth 150 likes”) are derived from the 2023 `twitter/the-algorithm` release and should be treated as directionally informative but not authoritative for the 2026 system.

The repository also excludes the protobuf schema crates (`xai_*_proto`), meaning the gRPC interface contracts between services are not directly inspectable from the public code, though they can be inferred from the service implementation files.

2.2 Thunder: Real-Time In-Network Post Store

Thunder (`thunder/`) is the Rust service responsible for the **in-network** arm of candidate sourcing, that is, it provides posts from accounts the requesting user already follows.

The canonical problem Thunder solves is one of pure latency engineering. At X’s scale, querying a persistent database (even an in-memory key-value store like Redis) for every feed request introduces too much I/O overhead to meet the sub-200-millisecond total budget. Thunder’s solution is conceptually simple but operationally demanding: **load all recent in-network post data directly into RAM**, maintain it incrementally in real time, and serve it via an in-process gRPC call that requires no network round-trip to an external store.

The mechanism works as follows. Thunder subscribes to a **Kafka topic of post-create and post-delete events**. As authors publish or delete posts anywhere on the platform, those events flow through Kafka and Thunder updates its in-memory data structures accordingly. Per-user, Thunder maintains ordered lists of:

- Original posts (full-text posts, not replies)
- Reply posts from the user

- Video posts from the user

When a feed request arrives in Home Mixer, it dispatches a gRPC call to Thunder's `InNetworkPostsService`. Thunder walks the in-memory lists for every account the viewer follows and returns a merged, recency-ordered candidate set, without hitting any external database whatsoever. The result is **sub-millisecond lookup latency** for a candidate pool that would otherwise require hundreds of database queries.

The trade-off is memory consumption. Thunder must hold recent post data for every active author in RAM simultaneously. The cost (in hardware) and the engineering effort (in keeping RAM consistent with the Kafka event stream) are non-trivial. For X's infrastructure budget and throughput requirements, however, this trade-off is clearly preferable to database round-trips.

From a creator's perspective, Thunder is the mechanism by which your followers see your posts. If you are a follower of an account, that account's recent posts are already sitting in Thunder, ready to be pulled on your next feed load. The ordering within Thunder's candidate pool reflects a combination of recency and the Phoenix ranker's predicted engagement score, Thunder surfaces the candidates, but the final ordering is determined downstream.

2.3 Phoenix Retrieval: Two-Tower Architecture and Approximate Nearest-Neighbor Search

Phoenix (`phoenix/`) performs the vastly more computationally complex task: discovering **out-of-network** posts, content from accounts the user does not follow, from a global corpus of hundreds of millions of recent posts.

The Phoenix retrieval stage uses a **two-tower transformer architecture** (a well-established pattern in industrial recommendation systems, pioneered by Google's dual-encoder models and YouTube DNN retrieval). The two towers are:

User Tower: Takes as input the viewer's demographic embedding, their recent engagement history (last N posts they liked, replied to, shared, dwelled on, quoted, or bookmarked, serialized as a sequence of action-weighted post identifiers), and their followed-topic metadata. This tower processes these inputs through a multi-layer transformer to produce a **normalized user embedding vector** of dimension D (256 in the mini Phoenix public model; larger in production).

Candidate Tower: Independently computes a normalized embedding for every post in the global corpus. Critically, these candidate embeddings are **pre-computed offline** and stored in a vector database. At Phoenix Retrieval time, only the user tower runs per-request; the candidate tower computation has already been done asynchronously when each post was published.

Similarity Search: The retrieval step computes **dot-product similarity** between the user embedding and all pre-computed candidate embeddings, then selects the Top-K closest candidates. This is an **Approximate Nearest-Neighbor (ANN)** search problem, because exhaustive search over hundreds of millions of embeddings is computationally intractable at feed-request latency. The repository references standard industrial ANN tooling, in practice at this scale, distributed vector stores such as **Milvus** or **Qdrant** (or proprietary equivalents) are used to shard the embedding space and execute sub-linear similarity queries.

The two-tower design has an important architectural consequence: the candidate embeddings are independent of the specific user requesting content. This means Phoenix can precompute candidate representations for every post as it is created, update the ANN index asynchronously, and serve retrieval

queries without ever needing to run the expensive candidate encoder at request time. Only the user encoder runs per-request, and it is a single forward pass through a transformer - manageable within latency constraints.

The output of Phoenix Retrieval is a set of out-of-network candidate posts ranked by embedding similarity, ready to be merged with Thunder's in-network candidates and passed to the Phoenix Ranker stage.

2.4 The GraphJet Legacy and Its Transformer Successor

GraphJet was a real-time graph-based recommendation engine introduced in the Twitter/X 2023 architecture. Its primary innovation, relative to the static **TweepCred** PageRank score (computed offline on a daily or weekly basis from the historical interaction graph), was the ability to reflect **dynamically evolving trust signals**, specifically, the graph of interactions occurring *at this moment*, measured in seconds.

In the 2023 system, GraphJet maintained a bipartite graph of users and tweets, updated in real time as new engagement events arrived. It could detect that, for instance, a user whose engagement vector was historically close to yours just replied to a post you haven't seen, implying an elevated likelihood that you would also find that post relevant. This was called **Inter-Niche Trust Propagation**: the observation that trust and interest signals can propagate across topical communities when users who bridge those communities engage with shared content.

The TweepCred PageRank score, by contrast, represented a slower-moving assessment of an account's credibility within the network. Accounts with high TweepCred (typically in the 65-100 range on a 0-100 scale) received broader organic distribution; accounts below the critical 0.65 threshold (normalized) had their post consideration sets severely restricted.

The **2026 Phoenix architecture fundamentally replaces both of these mechanisms**. The transformer model, ported from the Grok-1 architecture and adapted for recommendation use cases, learns to capture the functional equivalents of TweepCred and GraphJet's real-time trust propagation *implicitly* through training on the full user engagement history. Instead of maintaining a live graph and computing PageRank scores, Phoenix reads the user's action sequence (what they engaged with, when, and how strongly) and derives relevance from learned cross-sequence attention patterns.

This transition has a specific implication for how "authority" manifests in the new system. There is no longer a discrete TweepCred scalar that can be inspected or gamed in isolation. Instead, an account's effective authority is encoded in the embedding space that Phoenix has learned from aggregate engagement patterns: accounts whose posts reliably generate high-quality engagement signals (follows, DM shares, substantive replies) are implicitly assigned denser, more "reachable" positions in the embedding space. This is a smoother and less gameable representation of authority than the discrete PageRank-derived scalar it replaced.

That said, the **params/ module withheld from the public release** may retain certain learned biases toward credibility-proxying features (account age, verification status, Premium subscription) that function analogously to TweepCred. Field observations and the continued presence of a Premium visibility boost in the open-source code confirm that some explicit credibility signals persist.

2.5 Candidate Isolation: `make_recsys_attn_mask` and Complexity Reduction

One of the most technically distinctive decisions in the Phoenix ranking model is **Candidate Isolation**, a specific attention masking strategy implemented in `phoenix/grok.py` via the `make_recsys_attn_mask` function.

In standard transformer attention, every token (or, by analogy, every candidate post) can attend to every other token in the input sequence. In a naive recommendation ranking transformer, this would mean that when Phoenix scores Candidate A for User U, the model would consider the content and engagement signals of all other candidates (B, C, D... N) in the same batch, because they all attend to each other in the attention layer.

This creates two severe problems:

Problem 1, Computational complexity: If there are M candidates and L user history tokens, full candidate-to-candidate attention has complexity $O(M^2 \times L)$, because every candidate attends to every other candidate, and each candidate also attends to all L user history tokens. At $M=1500$ candidates and $L=128$ history tokens, this is computationally prohibitive at request latency.

Problem 2, Score instability: If Candidate A's score depends on which other candidates happen to be in the batch, then the same post would receive different scores in different requests (depending on what other candidates were retrieved alongside it). This makes the ranking non-deterministic and effectively un-cacheable.

The `make_recsys_attn_mask` function solves both problems simultaneously with an elegant masking strategy: **each candidate can attend to the user's engagement history and to itself, but cannot attend to any other candidate.** The attention mask is a binary matrix where:

- `mask[candidate_i, user_history_j] = 1` (candidate i can attend to user history token j)
- `mask[candidate_i, candidate_i] = 1` (candidate i can attend to itself)
- `mask[candidate_i, candidate_j] = 0` for $i \neq j$ (candidate i cannot attend to any other candidate)

This reduces the effective complexity from $O(M^2 \times L)$ to $O(M \times L)$, a linear rather than quadratic relationship with the number of candidates. At 1,500 candidates and 128 history tokens, this is a reduction from approximately 288 million attention operations to approximately 192,000. The speed gain enables Phoenix to rank all candidates within the latency budget of a live feed request.

The isolation property also makes each candidate's score **fully deterministic and cacheable**: given the same user history and the same candidate post, Phoenix will always produce the same engagement probability predictions, regardless of what other posts were retrieved in the same batch. This property is exploited aggressively in production caching strategies.

The `phoenix/README.md` explicitly flags this as a *critical design choice*: "The ranking model uses a transformer architecture where candidates cannot attend to each other during inference. This is a critical design choice that ensures the score for a candidate doesn't depend on which other candidates are in the batch."

2.6 Grox: The Content-Understanding Pipeline

Grox (`grox/`) is a Python-based service introduced in the **May 15, 2026** update. It is described in the repository as "a content-understanding pipeline that provides classifiers and embedders for tasks such

as spam detection, post-category classification, and PTOS (Public Town Hall Open Standard) policy enforcement.”

Architecturally, Grox is a worker pool of Vision-Language Model (VLM) inference processes that run **before** and **in parallel with** the main Phoenix retrieval-ranking pipeline. When a post is created anywhere on the platform, Grox processes it asynchronously to produce:

- **Spam classification score**, probability that the post is spam, operating on text, image, and video modalities simultaneously. Notably, `task_spam_detection.py` within Grox segments the user population by follower count before applying classifiers (discussed in detail in Section 5.6).
- **Safety and policy classification**, whether the post triggers PTOS violations, NSFW flags, or Community Notes signals (the `NSFA_COMMUNITY_NOTE` flag, added in 2026, now feeds directly into the `MediumRisk` bucket that throttles distribution).
- **Banger quality score**, a post-quality assessment combining a continuous `quality_score` (0.0–1.0) and a complementary `slop_score` (measuring low-quality content markers). Posts with `quality_score < 0.4` are demoted before reaching the Phoenix ranker (see Section 5.8).
- **Reply ranking score**, for replies in conversation threads, Grox applies a dedicated VLM prompt to assess reply quality on a 0–3 scale, with special treatment for replies directed at accounts with large followings. The May 2026 repo README describes “a dedicated prompt for replies to large accounts.”
- **is_image_editable_by_grok flag**, a novel signal introduced in the May 2026 update, marking whether a post’s image is something Grok can interpret and potentially re-caption or summarize for improved recommendation context.
- **Multimodal embeddings (v5)**, Grox generates unified content embeddings across text, image, audio, and video modalities using a v5 embedding model. These embeddings are the representations that get stored in the ANN index for Phoenix Retrieval. In the 2023 system, visual content had reduced representation in the embedding space; the v5 multimodal model gives all media types full parity with text in candidate generation.

Every post begins its life in the system at a **MediumRisk** classification, remaining in that state until Grox processes it and assigns a final safety clearance. This means that immediately after publication, before Grox has run, a post’s distribution is partially throttled. This represents an intentional “safe default” posture: assume a post might be harmful until the VLM has verified it is not.

2.7 Home Mixer: The Orchestration Layer

Home Mixer (`home-mixer/`) is the Rust-based gRPC server that coordinates the entire pipeline on behalf of a single feed request. It implements the `HomeMixerServer` and dispatches to two primary pipelines: `PhoenixCandidatePipeline` (for scoring individual posts) and `ForYouCandidatePipeline` (for full feed assembly including ads and module injection).

A single For You feed request triggers the following sequence within Home Mixer:

Stage 1, Query Hydration (21 parallel hydrators): Before any candidates are retrieved, Home Mixer hydrates the viewer’s context. This includes fetching: the user’s full follow list, mutual follow graph, muted accounts and keywords, blocked accounts, MinHash similarity signatures, impression bloom filters (recording which posts the user has already seen), served post history, recent engagement sequence, followed topics, starter pack memberships, demographic context, IP context, and subscription

status. These 21 hydrators run in parallel.

Stage 2, Candidate Sourcing (parallel): Thunder and Phoenix Retrieval run simultaneously. Thunder returns in-network candidates (from followed accounts); Phoenix Retrieval returns out-of-network candidates discovered via ANN search over the global embedding space. Additional candidate sources introduced in the May 2026 update include Phoenix MoE (Mixture of Experts variant), Phoenix Topics (interest-based retrieval), Prompts (conversational feed prompts), and Ads. All candidate sources run in parallel.

Stage 3, Candidate Hydration: Each candidate post is enriched with additional metadata: core post text, author information, engagement counts (likes, replies, reposts, views), brand safety signals, language code, media detection results, quote post expansions, mutual follow scores, and subscription status hydration.

Stage 4, Pre-Scoring Filters: Before the expensive Phoenix ranker is applied, a set of lightweight filters remove candidates that are clearly ineligible: posts from blocked or muted accounts, posts already seen (checked against the bloom filter), duplicates, posts flagged with MediumRisk safety classification, NSFW content for users who have not opted in, and stale posts beyond the relevance time window.

Stage 5, Scoring: The surviving candidates pass through the Phoenix ranker, which predicts engagement probabilities across 15-19 action types. The weighted scorer combines these predictions into a final scalar score. The author diversity scorer then applies a decay multiplier to prevent any single author from dominating a user’s feed.

Stage 6, Selection, Blending, and Side Effects: The top-K posts are selected, ads are injected at positions determined by the `home-mixer/ads/` module (respecting brand safety boundaries), “Who to Follow” modules are inserted, and the final response is returned. Side effects run asynchronously: impression logs are written, the served history bloom filter is updated, and request-level metrics are recorded.

3. Scoring Formula and Engagement Modeling

3.1 The Linear-Weighting Fallacy

Among the most pervasive misconceptions about X’s algorithm is the belief that it applies a simple, static linear formula to rank posts, something of the form:

$$\text{Score} = (\text{Likes} \times 30) + (\text{Retweets} \times 20) + (\text{Replies} \times 75) + \dots$$

This misconception originates from the **2023** `twitter/the-algorithm` release, which did expose fixed numerical weights for a subset of engagement signals in a component called the “Heavy Ranker.” Values such as +75.0 for an author replying back to a reply on their own post, or +0.5 for a standard like, were extracted from that codebase and have circulated widely as definitive algorithm truth ever since.

The 2026 Phoenix architecture makes these values **historically informative but architecturally obsolete**. xAI explicitly removed the concept of a hand-tuned weight table: “We have eliminated every single hand-engineered feature and most heuristics from the system.” The relationship between engagement signals and final ranking score is no longer a deterministic arithmetic formula; it is the **learned output of a multi-layer transformer** whose parameters encode billions of implicit relationships observed across hundreds of millions of user-post interaction sequences.

The critical distinction is **non-linearity**. In a linear formula, doubling the number of likes doubles

the score contribution from likes. In Phoenix’s transformer ranker, the relationship is fundamentally non-linear: the score contribution of any individual signal depends on the entire constellation of other signals present on the post, the viewer’s engagement history, the temporal context, the content of the post as understood by the Grox VLM, and learned interactions between all of these features. There is no clean arithmetic decomposition.

3.2 Phoenix Ranker: Multi-Head Non-Linear Scoring

The Phoenix Ranker is a transformer model ported from the **Grok-1 open-source release** by xAI and adapted for recommendation use cases. Its architecture consists of:

- **Input embeddings:** The model takes as input a concatenated sequence of the viewer’s engagement history (quantized as action-type embeddings + post embeddings + author embeddings + temporal position embeddings) and the candidate post’s feature representation.
- **Transformer encoder layers:** Multi-head self-attention layers process the combined sequence. The candidate isolation mask (described in Section 2.5) is applied at each attention layer, ensuring candidates attend only to the user history and themselves.
- **Multi-task prediction heads:** After the final transformer layer, separate linear projection heads are applied to the candidate’s final hidden state to produce one probability logit per predicted action type.
- **Sigmoid activations:** Each logit is passed through a sigmoid function to produce a probability in $[0, 1]$.

The mini Phoenix model published via Git LFS uses 256-dimensional embeddings, 4 attention heads, and 2 transformer layers. The production model uses larger dimensions, more heads, and more layers - the exact configuration is not public.

The model is trained with a **multi-task learning objective**: it jointly minimizes cross-entropy loss across all predicted action types simultaneously. This means that optimizing the representation for “predict whether the user will like this post” also improves the representation for “predict whether the user will follow the author,” because these tasks share a substantial portion of the underlying signal structure. Multi-task training produces more robust and generalizable feature representations than training separate models for each action type.

3.3 The Fifteen-to-Nineteen Engagement Probability Heads

The Phoenix Ranker predicts a probability for each of the following engagement actions, drawn from the repository’s `phoenix/README.md` and the confirmed `ActionName` proto enum:

Index	Action	Nature	Signal Polarity
1	P(favorite)	Like	Positive
4	P(reply)	Reply	Positive
5	P(quote)	Quote tweet	Positive
6	P(repost)	Repost	Positive
-	P(click)	Link/card click	Positive
-	P(profile_click)	Profile visit	Positive

Index	Action	Nature	Signal Polarity
13	P(video_view)	Video quality view (VQV)	Positive (gated)
-	P(photo_expand)	Photo tap-to-expand	Positive
-	P(share)	Generic share	Positive
-	P(dm_share)	Share via Direct Message	Positive (high weight)
-	P(copy_link_share)	Copy-link share	Positive
11	P(dwell)	Dwell time (binary + continuous)	Positive
-	P(follow_author)	Follow the author	Positive (highest weight)
-	P(not_interested)	“Not interested” tap	Negative (heavy)
-	P(block_author)	Block	Negative (severe)
-	P(mute_author)	Mute	Negative (severe)
-	P(report)	Report	Negative (severe)

An important technical note on `P(video_view)` : this prediction is subject to a **gating condition** via `VQV_WEIGHT_ELIGIBILITY` . The video quality view weight is set to zero unless the video’s duration exceeds a minimum threshold (`MIN_VIDEO_DURATION_MS`). Field analysis suggests this threshold is approximately 60-120 seconds, with community consensus gravitating toward approximately 2 minutes as the full-weight eligibility floor. This gating explicitly defends against creators attaching stub video clips to posts in an attempt to game the video engagement signal.

Similarly, `P(dwell)` is actually modeled as **two separate heads**: a binary classification head predicting whether the user will dwell at all, and a continuous regression head predicting dwell duration in milliseconds. Both heads contribute to the final score, with the continuous head providing gradient-rich training signal from every non-trivial viewing event.

3.4 Weighted Scorer and Final Score Computation

The final score for a candidate post is computed in `weighted_scorer.rs` according to the formula:

$$\text{Final Score} = \sum (\text{weight}_i \times P(\text{action}_i)) + \text{NEGATIVE_SCORES_OFFSET}$$

where: - `weight_i` is the learned or configured importance weight for action type `i` - `P(action_i)` is the transformer’s predicted probability for that action - `NEGATIVE_SCORES_OFFSET` is a constant offset ensuring that scores remain in a consistent numerical range even when negative action probabilities dominate

The `weight_i` values live in the withheld `params/` module. What is known from the code structure is the **ordering** of signal importance: negative actions (block, mute, report) carry weights so large in magnitude that a single predicted block event can offset hundreds of predicted likes. Field analysis and the legacy 2023 release weights suggest that `P(follow_author)` carries the single largest positive weight, followed by DM share, followed by author-engaged replies.

After the Phoenix score is computed, two **post-scoring modifiers** are applied before final selection:

Author Diversity Scorer (`author_diversity_scorer.rs`): Applies a decaying multiplier to posts

from the same author within a single feed response. If the decay factor is approximately 0.7, the first post from Author A receives a multiplier of 1.0, the second post from Author A in the same response receives 0.7, the third receives 0.49, and so on, with a floor preventing complete suppression. This is not a daily cap, it is a per-request decay, meaning the same author's posts could appear in multiple sessions throughout the day without penalizing each other, as long as they don't all appear in the same single feed refresh.

Out-of-Network (OON) Adjuster: A separate modifier for Phoenix Retrieval candidates that calibrates the raw OON score to reflect the confidence level of the similarity-based retrieval.

4. Positive Signals: Actions That Drive Reach and Discovery

4.1 Dwell Time Maximization

Mechanism: `P(dwell)` binary classification + continuous regression head in Phoenix Ranker.

Dwell time, the duration for which a viewer's attention rests on a post in the feed, even without any explicit interaction, is one of the most powerful implicit engagement signals in the Phoenix model. Unlike likes, replies, or retweets, dwell time requires no conscious volitional act from the viewer; it is captured passively by the client application, which reports to the backend the amount of time the post's bounding box was visible on screen and whether the user paused scrolling.

The model predicts two distinct dwell probabilities. The first is a binary question: did the user stop and read this, or did they scroll past? The second is a continuous regression: if they did stop, how many milliseconds did they remain? Posts that consistently generate long dwell events - even without generating explicit engagements like likes, accumulate a strong positive signal that Phoenix interprets as evidence of genuine relevance to the viewer.

Practical implication: Content formats that structurally extend the time a viewer spends on a single post increase dwell signal. Long-form posts (the algorithm now favors single long-form posts over multi-tweet threads where the character limit permits, because a single post concentrates the dwell event rather than distributing it across multiple impression events), native video that runs automatically, image carousels requiring multiple swipes, and polls that prompt a micro-deliberation (the viewer reads the options and considers their vote before scrolling) all mechanically increase dwell time per post.

The Phoenix ranker's training data encodes these dwell patterns across hundreds of millions of interaction sequences, giving it the ability to predict, before any engagement occurs, whether a new post is the type of content that generates sustained viewing attention.

4.2 Media Thumb-Stop, Hold Rate, and the Banger Screen

Mechanism: `banger_initial_screen.py` (Grox), `VQV_WEIGHT` gating, `P(video_view)` head.

The **Banger Screen** is a Grox-side pre-filter that every post passes through before reaching the Phoenix ranker. The function `banger_initial_screen.py` evaluates the post's raw multimodal content and produces:

- `quality_score` (float, 0.0–1.0): A general content quality estimate.
- `slop_score` (float): A measure of low-quality or low-originality content markers.

- `is_image_editable_by_grok` (boolean): Whether Grok can interpret and annotate the visual content.

Posts with `quality_score < 0.4` fail the Banger Screen and are demoted significantly, they remain in the pipeline but receive reduced consideration by the Phoenix ranker. The `slop_score` contributes to the `P(not_interested)` negative signal, effectively giving posts that pattern-match known low-quality content a pre-emptive negative weight.

For video content specifically, the concept of a **Thumb-Stop Rate** (the proportion of viewers who stop scrolling when the video begins auto-playing) and **Hold Rate** (the proportion who continue watching beyond a minimum segment, often cited as 25% of total duration) are strong signals that the VQV prediction captures. A video that achieves a high hold rate in its first 1.5 seconds, the “thumb-stop” window, signals to the model that this content has exceptional attention-capture qualities, qualifying it as a “banger” candidate.

Practical implication: If you post a native video that holds 25%+ of viewers past the 25% duration mark and exceeds the minimum duration threshold (`MIN_VIDEO_DURATION_MS`), the `VQV_WEIGHT` fires at full strength, potentially unlocking the out-of-network Phoenix retrieval pool. Conversely, stub videos below the duration threshold receive `VQV_WEIGHT = 0` , meaning the post is treated as a text post for scoring purposes regardless of the video attachment.

4.3 Direct Message Shares

Mechanism: `P(dm_share)` separate prediction head, distinct from `P(share)` and `P(copy_link_share)` .

The May 2026 codebase makes an explicit architectural distinction between three share modalities:

- **Generic share button tap**, low specificity; the user may have shared to any destination.
- **Share via Direct Message** (`dm_share`), high specificity; the user deliberately sent this post to a specific person or group, a strong “I want you specifically to see this” signal.
- **Copy-link share** (`copy_link_share`), medium specificity; the user is pasting the link somewhere (another platform, a messaging app, email), indicating external curation intent.

The `dm_share` signal carries the highest weight among the three because it is the most intentional act of curation. Sending a post in a DM requires navigating to the DM composer, selecting a recipient, and explicitly delivering the content, actions that indicate strong conviction in the post’s value.

Mechanism implication for out-of-network reach: When a post accumulates a critical mass of `dm_share` events in a short time window, Phoenix Retrieval interprets this as a signal that the post’s content embedding is “reachable” from a broader population than just the original audience. DM shares effectively vote for a post’s embedding to be surfaced in ANN results for users whose engagement vectors are similar to the shares’ recipients.

Practical implication: Creating content that people genuinely want to send directly to specific individuals, useful reference material, insightful analysis, entertaining content people associate with a friend, is the highest-ROI signal strategy in the 2026 algorithm. Unlike likes (passive, often reflexive) or reposts (visible and potentially performative), DM shares are private and therefore unpolluted by social pressure, making them the closest proxy to genuine, high-intent organic curation in the system.

4.4 Bookmarks and Quote Tweets

Mechanism: $P(\text{favorite})$ head for bookmarks (saved as a high-weight sub-type), $P(\text{quote})$ head for quote tweets.

Bookmarks represent a user’s explicit declaration that a post merits future reference, a “save for later” action that signals high value without requiring the social visibility of a like or repost. Field analysis of the 2023 legacy weights placed bookmarks at approximately 10x the value of a standard like; the 2026 transformer almost certainly continues to assign bookmarks an elevated weight relative to passive likes, because bookmarks are definitionally high-intent actions (no social pressure or reflexive habit is likely to drive saving a post to a personal library).

The practical implication is content strategy: posts framed as **reference material** (data compilations, definitional frameworks, how-to guides, templates, checklists) naturally attract bookmark behavior. “Bookmark this” calls to action, when used authentically rather than manipulatively, explicitly orient the viewer’s attention toward the save action.

Quote tweets ($P(\text{quote})$) are structurally distinct from reposts because they require the quoting user to add their own commentary, making them a compound signal: the quoting user found the original content valuable enough to engage with intellectually, and they are exposing their own followers to it with explicit editorial endorsement. Quote tweets are therefore high-weight positive signals for the Phoenix User Tower’s representation of the original author.

4.5 Profile Visit-to-Follow Conversion

Mechanism: $P(\text{profile_click})$ and $P(\text{follow_author})$ heads; `following_replied_users_hydrator.rs` viewer threshold.

The sequence of (1) viewer sees post, (2) viewer taps author avatar or name to open profile, (3) viewer presses “Follow” is the **highest-value engagement signal chain** in the Phoenix model. The $P(\text{follow_author})$ prediction head carries the largest single positive weight among all action types, because a follow is a durable, long-lasting commitment that guarantees future feed interactions and expands the in-network candidate pool for the follower in perpetuity.

When a post drives a significant number of profile-to-follow conversions, its score in both the current feed request and future retrieval episodes increases non-linearly: each new follower re-weights the author’s content positively in the follower’s Thunder candidate pool, creating a compound distribution effect where each tier of new followers generates the next tier of reach expansion.

A separate piece of code, `following_replied_users_hydrator.rs`, gates the “**people you follow replied to this**” social-proof badge with a constant `VIEWER_FOLLOWERS_THRESHOLD: i64 = 1000`. This means that if a viewer has fewer than 1,000 followers, the social proof module, which shows the avatars of followed accounts who replied to a post, is not displayed. This is relevant not to the post author but to the *viewer* account: accounts with fewer than 1,000 followers do not receive this social-trust amplification in their feed, which may contribute to slower organic discovery for smaller accounts.

4.6 Self-Reply Bumping and Early Reply Velocity

Mechanism: Time decay function in candidate freshness scoring; `PostCandidate` timestamp metadata; reply velocity signal in early distribution tier.

The temporal dimension of X's ranking model is governed by a **time decay function** that reduces a post's base score as it ages. The exact form of this function is not published in the open-source code, but it is consistent with an exponential or polynomial decay curve, posts from the past few hours receive full freshness weight; posts from the past 24 hours receive reduced weight; posts older than several days are effectively retired from active distribution.

Field practitioners have observed a persistent behavior: posting a substantive reply to your own post within the first few minutes of publication effectively re-enters the post into the freshness window for a new wave of impressions. This is mechanistically consistent with how Home Mixer treats reply posts: a self-reply creates a new candidate event in Thunder (with a fresh timestamp), which, when surfaced, draws viewer attention back to the original post in the thread context.

More importantly, **early reply velocity** (the rate at which replies accumulate in the first 15-60 minutes after posting) is one of the strongest signals driving the post into the first out-of-network expansion tier. Phoenix's tiered distribution logic, while not explicitly published in the open-source code, is consistent with the following observed behavior: a post that accumulates replies above the author's baseline reply rate in the first 15 minutes qualifies for the first Thunder distribution expansion; if the reply rate holds above baseline, it qualifies for Phoenix Retrieval inclusion. Author-replied replies (where the original poster responds to a commenter) receive separate elevated scoring in Grox's reply ranker, further amplifying this signal.

4.7 Topical Authority and Embedding Stability

Mechanism: User Tower embedding consistency; Phoenix's learned representation of topical specialization.

In the 2023 system, topical authority was partially implemented through **SimClusters**, a system that assigned each user to one or more topical interest communities based on their engagement history, and used these cluster memberships to weight candidate retrieval. The 2026 system eliminates SimClusters and replaces the explicit cluster assignment with an implicit representation learned by the Phoenix User Tower.

The practical consequence, however, is similar: an account whose engagement history consistently centers on a narrow vertical (e.g., cybersecurity, mechanical keyboards, Argentine football) builds an **engagement-sequence embedding that converges toward a stable, distinctive region of the embedding space**. The User Tower learns to represent this account with a tightly clustered embedding that, when used for ANN retrieval, consistently pulls candidates from the same topical space, and, symmetrically, causes the account's posts to appear as ANN candidates in the feeds of users with similar topical embeddings.

Accounts that frequently pivot between unrelated verticals produce a diffuse, unstable embedding that is less likely to be pulled as a strong ANN match for any specific topical query. The Phoenix system does not penalize pivoting explicitly; it simply fails to build the strong embedding specificity that drives out-of-network discovery.

Practical implication: Sustained topical focus over 60+ days produces measurable improvement in out-

of-network reach for content within that vertical. The improvement is not rule-based; it is an emergent consequence of the User Tower’s embedding dynamics. Diversifying abruptly dilutes the embedding and temporarily reduces OON discoverability, though the effect recovers as the new topical history accumulates.

4.8 X Premium Effect on In-Network and Out-of-Network Visibility

Mechanism: Premium status flag in user and author candidate hydrators; TweepCred-adjacent credibility signal.

X Premium (the paid subscription tier including Blue, Gold for businesses, and Gray for government) confers measurable algorithmic benefits confirmed by multiple analyses of the open-source code and field observations:

- **4x increase in in-network visibility** (reach to followers) relative to comparable non-Premium accounts with equivalent engagement profiles.
- **2x increase in out-of-network visibility** (reach via Phoenix Retrieval to non-followers).
- **Partial mitigation of link reach penalties** (Premium accounts posting external links suffer less severe reach suppression than free accounts, as discussed in Section 5.1).
- A TweepCred-equivalent credibility signal that reduces the friction applied by default to less-established accounts.

These benefits are **multiplicative on existing engagement potential**, not additive. A Premium account posting low-quality content that fails the Banger Screen or generates high $P(\text{not_interested})$ predictions does not get algorithmically rescued by Premium status; the multiplier amplifies the existing score rather than replacing it.

Importantly, the specific numerical coefficients for the Premium boost (4x in-network, 2x OON) are derived from field analyses and xAI’s own public statements, not from the open-source code directly (the relevant weights are in the withheld `params/` module).

5. Negative Signals: Actions That Suppress Reach and Trigger Penalties

5.1 External Links

Mechanism: Platform retention objective; inferred link penalty in hydration and scoring; possibly the Grox content classifier’s implicit down-weighting.

The relationship between external links and reach suppression on X has intensified significantly since 2023. The 2023 `twitter/the-algorithm` code explicitly applied a link penalty to posts containing outbound URLs, as confirmed by the code. Field observations document the progression:

- **2023:** 20–30% reach reduction for posts with external links.
- **2025:** 30–50% reach reduction; non-Premium accounts with links begin seeing near-zero median engagement in many verticals.
- **March 2026 and onwards:** Non-Premium accounts posting external links in the main post body now see median engagement drop effectively to zero in many cases.

The architectural rationale is clear from X’s stated platform objective: the system explicitly favors con-

tent that maximizes time spent on X. A post containing an external link is, from the system’s perspective, an exit ramp from the platform. The algorithm’s optimization objective (maximize engagement signals that correlate with session time, dwell, replies, follows, reposts) is directly undermined by content that sends users elsewhere.

An important caveat: the **2026 open-source release does not contain an explicit hard-coded link penalty** as a discrete weight or filter. The analysis at Postory.io notes: “There’s no hard-coded link penalty in the open code.” This does not mean the penalty does not exist, it means it is either encoded in the withheld `params/` module, embedded implicitly in the model’s learned weights (posts with links historically receive less engagement → the model learns to predict lower engagement for link-containing posts), or applied upstream through the Grox content understanding phase.

Practical implication: Place external links in the **first comment** rather than the main post body. This is the most widely cited field workaround and is consistent with the architectural incentive: a post without a link in its body is judged solely on its content quality and engagement potential. The link in the reply ensures users can find it, without triggering the in-body link suppression.

5.2 Hashtag Inflation

Mechanism: Grox spam classifier; pattern-based spam detection signals in `task_spam_detection.py`.

Hashtag behavior represents one of the clearest examples of a signal that was positively valued under the 2023 manual-feature architecture and is now actively penalized by the 2026 transformer and Grox classifier.

In X’s early recommendation system, hashtags were a topical indexing mechanism: adding hashtags helped the system assign a post to relevant SimCluster communities. In the 2026 system, topical assignment is learned implicitly from post content and engagement sequences by Phoenix, explicit hashtag signals are largely redundant and, when used in excess, become a **spam behavioral marker**.

The Grox `task_spam_detection.py` classifiers flag high-hashtag-density posts because they pattern-match known spam and low-quality content behaviors. Posts with 3 or more hashtags are consistently associated in training data with engagement-baiting, spam, and low genuine interaction rates. The classifier assigns a higher spam probability to these posts, which feeds into the Grox quality score and reduces Phoenix ranker consideration.

Field analyses cite approximately **40% reach reduction** for posts with 3+ hashtags. The architectural mechanism is not a binary rule (“3 hashtags = spam”) but a learned probability gradient: at 0-1 hashtags, spam probability is low; at 3+ hashtags, spam probability increases substantially. The exact threshold varies by topic, account history, and post content.

Practical implication: Use at most 1-2 highly relevant hashtags, or none at all. The Phoenix system now discovers topical relevance from content semantics rather than hashtag metadata. Adding hashtags primarily risks triggering Grox’s spam classifier without providing meaningful topical indexing benefit.

5.3 Burst Posting and Author Diversity Decay

Mechanism: `author_diversity_scorer.rs` ; per-request decay multiplier.

The `author_diversity_scorer.rs` module implements a geometrically decaying multiplier that ap-

plies to multiple posts from the same author appearing in a **single feed response**. The decay formula is approximately:

```
fn multiplier(&self, position: usize) -> f64 {
    (1.0 - self.floor) * self.decay_factor.powf(position as f64) + self.floor
}
```

Where `position` is the ordinal position of this author's posts within the feed response (0-indexed), `decay_factor` is approximately 0.7, and `floor` prevents the multiplier from reaching zero completely.

The first post from Author A in a given feed response receives `multiplier = 1.0`. The second post receives `multiplier ≈ 0.7`. The third receives `multiplier ≈ 0.49`. The fourth `≈ 0.34`, and so on. Each additional post from the same author in the same response is scored at roughly 70% of the previous post's effective score.

An important precision: this is a **per-request** decay, not a per-day cap. If you post 10 times in a day, the algorithm does not cap your daily distribution. What it does is ensure that multiple posts from the same author are unlikely to all appear in the **same feed refresh**. If a viewer opens X once at 9 AM and once at 6 PM, your 10 posts can appear across those two sessions without inter-penalizing each other, as long as no single session shows too many of your posts consecutively.

Practical implication: The commonly cited “post 2-3 times per day” advice does not reflect the actual per-request mechanism. The correct optimization is to **space posts across the day so that no single viewer's feed refresh encounters multiple of your posts in the same response**. A creator posting 5 times at 5-minute intervals faces a more severe diversity decay (because a single session of a fast-refreshing viewer will encounter all 5 posts) than one posting 5 times at 2-hour intervals (where different refresh events encounter different single posts).

5.4 Grox Sentiment Loop Penalty (Conversational Gaming)

Mechanism: Grox reply quality classifier (0–3 scale); spam loop pattern detection; `P(not_interested)` prediction amplification.

A well-documented manipulation strategy in earlier X algorithms was the **Reply Chain Exploit**: an author and their allies would post rapid-fire low-quality replies (“Thanks!”, “Exactly!”, “Haha”, “So true”) on each other's posts in the first few minutes after publication, artificially inflating the reply count signal and driving early distribution.

The Grox reply ranking classifier explicitly targets this behavior. Its 0–3 quality scale assigns: - Score 3: substantive, on-topic, adding new information or a meaningful perspective. - Score 2: positive but generic, affirmative without adding content. - Score 1: short, contextless, or purely reactive. - Score 0: spam-pattern content, including low-diversity reply loops.

Critically, Grox's classifier is a **Vision-Language Model reading the actual text of the reply**, not just its length or engagement velocity. A 4-word reply of “This changed my life” is evaluated on its semantic relationship to the original post, not merely its brevity.

When a post's reply graph consists predominantly of low-quality Grox scores (i.e., score 0-1 replies from the same small cluster of accounts appearing at high frequency), the composite engagement signal becomes contaminated: the high reply count is no longer reliably interpreted by Phoenix as genuine

interest signal. Additionally, the replies' collective `P(not_interested)` predictions aggregate and contribute negatively to the post's score.

Reply engagement pods that operated effectively under the 2023 architecture, where reply count was treated as a raw positive signal independent of reply quality, are explicitly defeated by Grox's semantic quality scoring in 2026.

5.5 Grox Toxicity and Brand Safety Filter

Mechanism: Grox PTOS classification; Brand Safety signal in `home-mixer/ads/`; `P(not_interested)` and `P(report)` prediction amplification over time.

The Grox content pipeline includes a **toxicity and aggressiveness classifier** that evaluates the sentiment polarity and adversarial character of post content and historical posting patterns at the account level. This is distinct from the post-level spam screen, it operates on the **cumulative content profile** of an account.

xAI's January 2026 release confirmed that Grok monitors the tone of every post. The ranking implications are stated explicitly: positive and constructive messaging receives wider distribution; negative and combative tones lead to reduced visibility, even when engagement metrics are high.

The Brand Safety system in `home-mixer/ads/` provides additional downstream pressure on high-toxicity accounts: advertisers purchasing placements in the X feed can specify brand safety categories, and the `home-mixer/ads/` module will not place their ads adjacent to posts flagged with certain content signals. An account that consistently scores high on toxicity classifiers becomes **less advertiser-compatible**, reducing the economic incentive for X to distribute its content in ad-supported placements. While this does not directly suppress the organic recommendation score, it removes a class of distribution surfaces that ad-adjacent content can reach.

Field observations suggest that accounts with persistently high toxicity profiles can see **out-of-network reach reductions of 70%+**, effectively limiting their distribution to their existing follower base via Thunder only.

5.6 Sub-1,000 Follower Account Filter: The `lte_1000` Bucket

Mechanism: `task_spam_detection.py` follower-count segmentation; dedicated VLM spam screen for small accounts.

The `task_spam_detection.py` file within the Grox module implements a **follower-count-based segmentation** for spam detection that applies materially higher scrutiny to smaller accounts:

```
# Follower count buckets in task_spam_detection.py:
# lte_100    - accounts with ≤100 followers
# lte_500    - accounts with ≤500 followers
# lte_1000   - accounts with ≤1,000 followers
# gt_1000    - accounts with >1,000 followers
```

The critical behavior for the `lte_1000` bucket: when a small-account user's reply is processed by Grox, the spam VLM runs an **additional screening pass** that larger accounts do not receive. When this secondary screen flags a reply as spam, Grox logs a specific metric counter:

"Reply Spam Found for lower than 1000 follower bucket." This metric counter is a **named, dedicated instrument in the codebase**, it is not an incidental log line, but a deliberately monitored signal.

The practical implication is severe for small accounts attempting to grow through reply strategies. Mass-replying from a fresh or small account runs directly into a filter that is explicitly named after and designed for that demographic. The `lte_1000` VLM screen is more conservative than the `gt_1000` screen, meaning the same reply content from a small account is more likely to be flagged as spam than the identical content from a large account.

This is compounded by the `following_replied_users_hydrator.rs` viewer threshold (Section 4.5): small accounts not only face higher spam screening on their own replies, but their followers also do not see the “people you follow replied to this” social proof badge.

The path through the `lte_1000` bucket: The algorithm does not suppress small accounts permanently or categorically. The filter targets **patterns**, specifically high-frequency replies from accounts with thin engagement histories. A small account posting fewer, higher-quality, contextually relevant replies (scoring 2-3 on Grox’s reply quality scale) is demonstrably less likely to be flagged than one posting many generic or repetitive replies.

5.7 Unfollow Spikes and Follow/Follower Ratio Imbalance

Mechanism: `P(not_interested)` signal accumulation; author-level engagement credibility; TweepCred-equivalent stability signal.

When a post drives an anomalous spike of **unfollows** immediately following its publication, the system receives a compounded negative signal: each unfollow event is an explicit declaration by a follower that they no longer want this author’s content, which is semantically equivalent to a strong `P(not_interested)` prediction being retroactively confirmed. When many followers simultaneously make this decision after a single post, the system updates the author’s effective engagement credibility downward, a durable signal that persists beyond the immediate post.

Field analyses document a **reach suppression window** of approximately 3 months following a severe unfollow spike, during which the author’s posts receive reduced OON consideration in Phoenix Retrieval. This suppression reflects the system’s Bayesian-like update: prior evidence of strong disengagement from a post increases the prior probability that future posts from the same author will also generate disengagement.

Follow/Follower ratio imbalance, specifically, following a very large number of accounts while being followed by very few, functions as a proxy spam signal in the TweepCred-equivalent components of the 2026 system. Accounts with extremely high follow-to-follower ratios (following 10,000 accounts while being followed by 50) pattern-match bot and engagement-pod behaviors associated with follow-for-follow schemes, which the system is trained to down-weight.

5.8 Visual Recycling and the Banger `slop_score` Penalty

Mechanism: Grox VLM visual content analysis; `slop_score` from `banger_initial_screen.py`; `original_media_flag` in candidate metadata.

The Grox Vision-Language Model evaluates the visual content of images and video thumbnails for **originality and novelty**. Content that pattern-matches previously seen, widely distributed, or stock imagery accumulates a higher `slop_score`, Grox’s internal measure of low-originality content.

The underlying mechanism is that the Grox VLM has been trained on a corpus of known stock photo libraries, widely-shared viral image templates, and screenshot-heavy content (text screenshots, captioned meme formats). When a post’s visual content matches these known-low-originality patterns, the `slop_score` component of the Banger Screen’s evaluation increases, which in turn reduces the composite `quality_score` and increases the probability that the post fails the 0.4 quality threshold.

The **original_media_flag** is set as part of the candidate hydration phase and signals that a post’s media content was originally created (not reposted from another platform or lifted from stock libraries). While the exact weight attached to this flag is not public, the contrast is consistent: original photography, original video captured by the author, and original audio content outperform recycled visual content in Grox’s quality assessment.

Practical implication: Screenshots of other platforms’ content, reposted viral imagery, stock photos from standard libraries, and heavily recycled meme templates are all likely to score poorly on Grox’s `slop_score`. Original visual content, even imperfect original content, is evaluated on substantive quality metrics rather than being pre-penalized for visual unoriginality.

5.9 YouTube and External Video Player Links

Mechanism: `VQV_WEIGHT` gating; `P(video_view)` head ineligibility for external players; `MIN_VIDEO_DURATION_MS` gate.

The `VQV_WEIGHT` (Video Quality View weight) in Phoenix’s weighted scorer is one of the highest positive weights in the engagement scoring system. It rewards posts whose videos generate sustained viewing above the duration eligibility threshold.

This weight applies **exclusively to native X video uploads**. Posts containing links to YouTube, Vimeo, TikTok, or any other external video platform are treated as link posts, not video posts - they do not trigger the video view prediction head, do not benefit from `VQV_WEIGHT`, and do not accumulate the auto-play dwell signal that native video generates. The viewer’s click on a YouTube link exits the platform entirely, generating an exit event that the platform’s session model registers negatively.

The asymmetry is total: a 3-minute native video that achieves a 40% hold rate qualifies for `VQV_WEIGHT` at full strength and unlocks OON distribution via Phoenix Retrieval. An identical video hosted on YouTube, linked in the same post, receives `VQV_WEIGHT` = 0, generates an external-link penalty on the post’s body, and sends the viewer off-platform.

Practical implication: Any video content intended for serious algorithmic distribution on X must be uploaded natively. The additional friction of uploading to both YouTube and X natively is, from a pure algorithmic optimization perspective, a non-negotiable cost of reaching X’s algorithm.

6. Temporal and Complexity Analysis

End-to-End Latency Budget

The system as a whole is designed to return a scored, ranked, filtered, and blended For You feed in under 200 milliseconds per request. This budget is allocated approximately as follows across the pipeline stages:

- **Query hydration (21 parallel hydrators):** ~10-30 ms (parallel I/O bounded)
- **Thunder in-network candidate retrieval:** <1 ms (in-memory, per design)
- **Phoenix Retrieval ANN search:** ~20-50 ms (ANN index lookup over distributed vector store)
- **Candidate hydration:** ~10-30 ms (parallel metadata fetching)
- **Pre-scoring filters:** <5 ms (Bloom filter lookups, eligibility checks)
- **Phoenix Ranker inference:** ~30-80 ms (transformer forward pass, batched)
- **Author diversity scoring and OON adjustment:** <5 ms
- **Ad blending and feed assembly:** ~5-15 ms
- **gRPC serialization and network:** ~5-20 ms

The Phoenix Ranker, despite being the computationally most expensive step, fits within the latency budget because of the candidate isolation masking optimization ($O(M \times L)$ complexity vs. $O(M^2)$ naive), aggressive batch inference with pre-computed candidate embeddings (the candidate tower runs offline), and efficient JAX/XLA compilation for matrix operations on accelerator hardware.

The candidate isolation property (Section 2.5) is particularly valuable here because it makes all candidate scores fully independent, enabling the ranker to process all candidates in a single batched forward pass with no sequential dependencies between them.

Scale Characteristics

- **500 million posts per day** ingested and indexed.
- **~1 billion candidate posts** in the active corpus at any given time.
- **~1,500 candidates per user** selected for full Phoenix ranking per feed request.
- **5 billion ranking decisions per day** across the user base.
- The funnel ratio is approximately **700,000:1**, the system discards 699,999 posts for every one it actually shows to a user.

The pipeline achieves this scale through a combination of architectural decisions: the two-tier candidate sourcing (cheap Thunder in-memory lookup + expensive ANN search only for OON) ensures that the expensive retrieval work is proportional to discovery needs, not to the total post corpus. The offline pre-computation of candidate embeddings ensures that the marginal cost per request includes only user tower computation and ANN lookup, not candidate encoding.

7. Security, Adversarial Risk, and the Open-Source Paradox

The Transparency Commitment

In announcing the January 2026 open-source release, xAI committed to a **4-week public update cadence**, a substantially more aggressive transparency posture than any prior social media recommendation system. The May 15, 2026 update was the largest single-commit update to the repository (187 files changed, ~18,000 lines of new code) and was delivered on schedule.

The stated rationale for transparency is to enable public accountability for the feed algorithm that shapes

information consumption for hundreds of millions of users. The Apache 2.0 license permits unrestricted use, modification, and redistribution of the code, including commercial use.

Adversarial Risks from Transparency

Publishing the full recommendation architecture creates a well-characterized **adversarial attack surface**. Researchers and practitioners with direct access to the pipeline code, signal definitions, and (through the mini Phoenix model) approximate weight magnitudes can reverse-engineer optimization strategies for artificial reach amplification.

The specific risks include:

Engagement pod optimization: Knowing the exact action types and their approximate relative weights enables sophisticated engagement pods to target high-weight actions (DM shares, follows, author-replied replies) rather than low-weight actions (likes). The 2026 system defends against this through Grox’s semantic quality classifier and the pattern detection in `task_spam_detection.py`, but the defense is probabilistic, not absolute.

VLM adversarial inputs: The Grox VLM performs visual and text safety classification. Adversarial prompting techniques developed for large language models may be applicable to elicit false negatives from Grox’s safety classifiers, images or text crafted to pass safety screens while containing policy-violating content.

Embedding space exploration: The public mini Phoenix model enables researchers to probe the embedding space directly, potentially discovering content patterns that consistently achieve high similarity scores for specific user embedding clusters.

The withheld `params/` Module

The deliberate exclusion of the `params/` module (production weight values) from the open-source release represents xAI’s primary defense against the most direct optimization attacks. Without the exact weights, adversarial optimization requires either training against the public mini model (which uses different, non-production weights) or empirical A/B testing at scale (which is noisy and expensive). This creates a meaningful but imperfect barrier.

Community Notes as a Ranking Signal

The May 2026 update confirmed that Community Notes impact feed distribution. The `NSFA_COMMUNITY_NOTE` flag (Notes Suggested for Amplification, Community Note) is now in the **MediumRisk list** for candidate filtering. Posts that receive Community Notes suggesting they contain misleading information are demoted in distribution, representing a rare explicit use of community-sourced editorial signals in the ranking architecture.

8. Reproducibility Framework

The May 15, 2026 update introduced the first end-to-end runnable inference pipeline for the public algorithm. The reproducibility framework centers on the `phoenix/run_pipeline.py` entry point and the pre-trained mini Phoenix model artifacts.

Artifact Structure

The ~3 GB Git LFS archive (`oss-phoenix-artifacts/`) contains:

```

oss-phoenix-artifacts/
├─ retrieval/
│   ├── model_params.npz      # Retrieval transformer + candidate tower weights (3 MB)
│   ├── embedding_tables.npz  # User/item/author hash embeddings, 1M each (1.4 GB)
│   └─ config.json           # Model config + hash function parameters
├─ ranker/
│   ├── model_params.npz      # Ranking transformer + action head weights (3 MB)
│   ├── embedding_tables.npz  # User/item/author hash embeddings, 1M each (1.4 GB)
│   └─ config.json           # Model config + hash function parameters
├─ sports_corpus.npz         # 537,000 sports posts with pre-computed candidate embeddings
└─ example_sequence.json     # Example user action history (3 posts: NFL, NBA, NHL)

```

Running the Pipeline

```

# Install dependencies via uv
uv sync

```

```

# Run end-to-end retrieval → ranking
uv run phoenix/run_pipeline.py --artifacts_dir artifacts/oss-phoenix-artifacts

```

This executes: 1. Loading the mini Phoenix retrieval model and the sports corpus embeddings. 2. Encoding the example user sequence from `example_sequence.json`. 3. Running ANN retrieval over the 537K sports corpus embeddings. 4. Passing retrieved candidates through the Phoenix ranker. 5. Outputting ranked results with predicted engagement probabilities per action type.

Customizing the User History

The `example_sequence.json` file contains three sample sports-post interactions (NFL, NBA, NHL). Researchers can substitute their own action sequences to observe how different engagement histories shift the retrieval and ranking results. Each history item requires: - `post_id` : Identifier for the engaged post. - `author_id` : Author of the engaged post. - `actions` : Dictionary mapping action index to value, using the `ActionName` enum: - 1 = favorite (like) - 4 = reply - 5 = quote - 6 = repost - 11 = dwell - 13 = video quality view (VQV)

Limitations of the Demo Environment

The mini Phoenix model is explicitly not the production model. It uses smaller dimensions (256 vs. production's larger configuration), fewer transformer layers, and is trained on a non-public subset of user interactions. The demo corpus (`sports_corpus.npz`) contains 537K sports posts, a tiny fraction of the full platform corpus.

The most critical limitation: the training dataset is not distributed, for privacy reasons. The demo model arrives pre-trained, so researchers can observe its inference behavior but cannot reproduce or retrain from scratch. Anyone seeking to build an independent production-equivalent recommendation system using this codebase must collect their own training dataset.

9. Conclusion

The transition from the 2023 Scala-based `twitter/the-algorithm` to the 2026 Rust-and-Python `xai-org/x-algorithm` represents more than an infrastructure modernization. It constitutes a fundamental philosophical shift in how relevance is defined, discovered, and enforced in one of the world’s largest real-time information systems.

The central thesis of the new architecture, stated without qualification in the repository README - is that *learned representations of engagement history are more powerful than any set of hand-engineered features*. The Phoenix transformer does not know that “DM shares indicate high intent” or that “burst posting triggers diversity decay” as explicit programmatic rules. It has learned the functional equivalents of these rules from billions of user-post interaction sequences, in a form that is simultaneously more robust (resistant to single-dimension gaming), more adaptive (updating continuously as platform behavior evolves), and less interpretable (no single weight or feature can be isolated and modified to produce a predictable outcome).

For practitioners and researchers, the implications are clear. The optimization strategies of the 2023 era, targeting specific numeric engagement weights, building hashtag strategies around topical indexing, gaming reply chains with pods, are architecturally obsolete. The 2026 system rewards content that generates the **full pattern of genuine engagement** that Phoenix has learned to recognize across 500 million posts per day: sustained dwell, high-quality substantive replies, DM-worthy value, organic follow conversions, and multimodal originality.

The open-source release, for all its deliberate incompleteness (withheld `params/` , withheld training data, mini model vs. production model), represents an unprecedented level of transparency in industrial recommendation systems. It enables independent verification of the pipeline’s architecture, meaningful public discussion of its incentive structures, and reproducible research on a genuine production recommendation system. The 4-week public update cadence, if maintained - will provide ongoing visibility into the system’s evolution.

The adversarial risks of this transparency are real but manageable within the current partial release framework. The withheld production weights are the correct boundary: sufficient opacity to prevent direct optimization attacks, sufficient transparency to enable accountability.

What the code ultimately reveals is a system optimized, without apology, for one objective: keeping users on X, engaging with content, for as long as possible per session. Every architectural decision - from Thunder’s sub-millisecond in-memory retrieval, to the Grox VLM’s brand-safety enforcement, to the external-link effective penalty, to the `P(follow_author)` head’s status as the highest-weight positive signal, is a consequence of that single optimization objective. Understanding this objective is the prerequisite to understanding why every individual mechanism in the system behaves the way it does.

Appendix A: Key Technical Abbreviations and Components

Term	Definition
ANN	Approximate Nearest Neighbor (similarity search algorithm)
Banger Screen	<code>banger_initial_screen.py</code> - Grox pre-filter scoring post quality
Grox	Python VLM content-understanding pipeline (spam, safety, quality)

Term	Definition
Home Mixer	Rust orchestration layer; gRPC server; assembles full feed
JAX	Google’s numerical computing library; used for Phoenix ML inference
LFS	Git Large File Storage; distributes mini Phoenix model (~3 GB)
lte_1000	Follower-count bucket in <code>task_spam_detection.py</code> for <1K accounts
MediumRisk	Default safety state of every new post until Grox clears it
Mini Phoenix	Public pre-trained model: 256-dim, 4 heads, 2 layers
OON	Out-of-Network (content from accounts the user does not follow)
Phoenix	Grok-1-based two-tower retrieval + transformer ranking system
PTOS	Public Town Hall Open Standard (X’s policy enforcement framework)
Thunder	Rust in-memory Kafka-backed in-network post store
TweepCred	Legacy (2023) PageRank-based credibility score; replaced in 2026 by learned representation
Two-Tower	Retrieval architecture with separate User Tower and Candidate Tower
VLM	Vision-Language Model; used by Grox for multimodal content analysis
VQV	Video Quality View score ($P(\text{video_view})$ head, gated by duration)
VQV_WEIGHT	Weight applied to video view prediction; 0 below <code>MIN_VIDEO_DURATION_MS</code>

Appendix B: Primary Source References

1. `xai-org/x-algorithm`, GitHub repository, Apache 2.0 license.
URL: <https://github.com/xai-org/x-algorithm>
2. Commit `e414c17`, May 15, 2026 major update (Grox, runnable pipeline, pre-trained model).
URL: <https://github.com/xai-org/x-algorithm/commit/e414c171ed68266341193330bc4864bf3f3534e3>
3. `phoenix/README.md`, Architecture documentation for Phoenix retrieval and ranking.
URL: <https://github.com/xai-org/x-algorithm/blob/main/phoenix/README.md>
4. `twitter/the-algorithm`, 2023 legacy release (Scala architecture, legacy engagement weights).
URL: <https://github.com/twitter/the-algorithm>
5. xAI Engineering Blog, January 20, 2026 announcement. `@XEng` on X.
6. DeepWiki analysis, `xai-org/x-algorithm` component documentation.
URL: <https://deepwiki.com/xai-org/x-algorithm>

Report compiled June 25, 2026. All architectural claims are grounded in the publicly released `xai-org/x-algorithm` codebase, field analyses from independent researchers, and xAI’s published statements. Specific production weight values (not published in the open-source release) are noted where they derive from legacy 2023 code or community field analysis rather than the 2026 codebase directly.